



University of Minho
School of Engineering



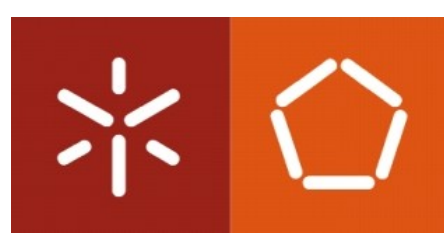
Cryptography & Network Security

Bachelor in Telecommunications and Informatics Engineering

João Marco Silva

Department of Informatics
joaomarco@di.uminho.pt

2024/2025



Stream Ciphers

Recap One-Time Pad

An unconditionally secure cipher, that:

- Uses a key as long as the message (*i.e.*, plaintext);
- The key is randomly generated;
- Each key is used to encrypt and decrypt a single message.

It produces random output that bears no statistical relationship to the plaintext.

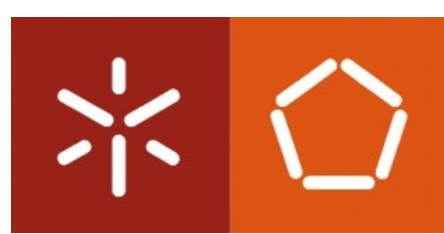
It usually operates over a binary alphabet using *XOR* operations.

$$C_i = p_i \text{ XOR } k_i$$

$$p_i = C_i \text{ XOR } k_i$$

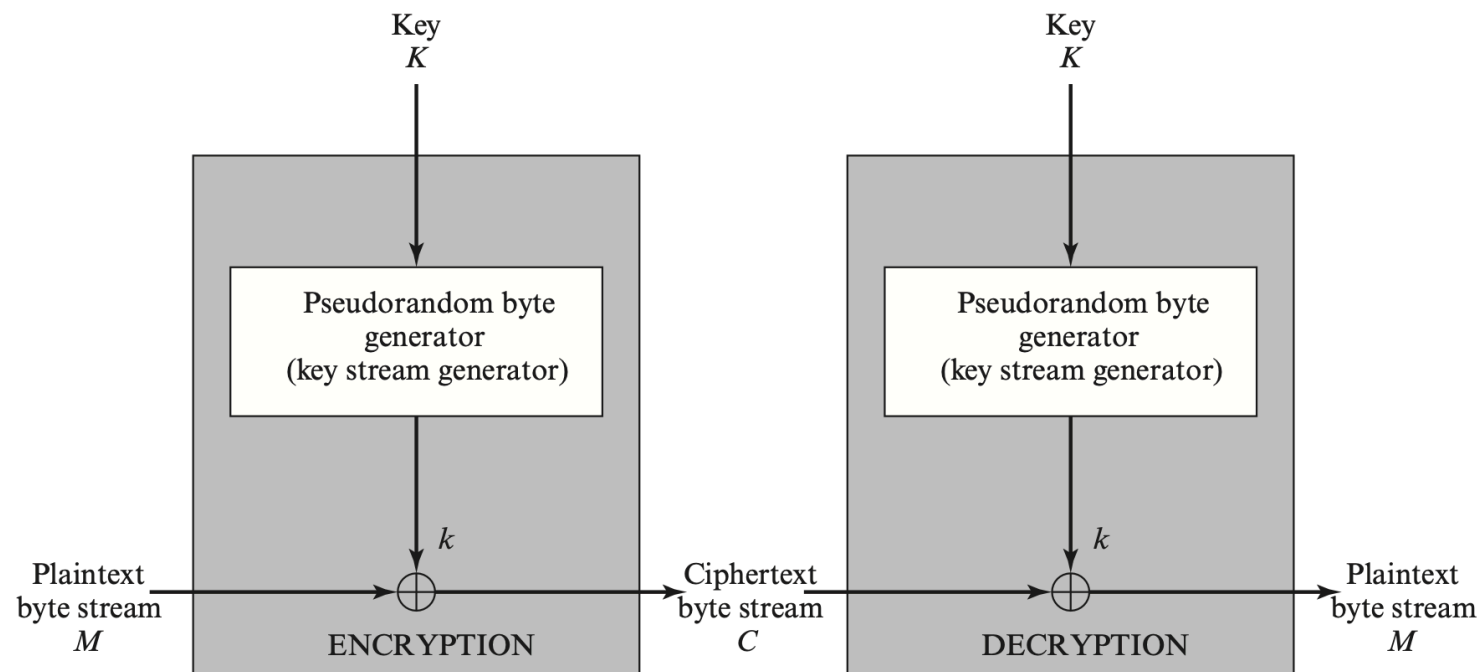
In practice, its security is entirely due to the randomness of the key. If the stream of characters (or bits) in the key is truly random, then so are the characters (or bits) in the ciphertext.

Stream Ciphers



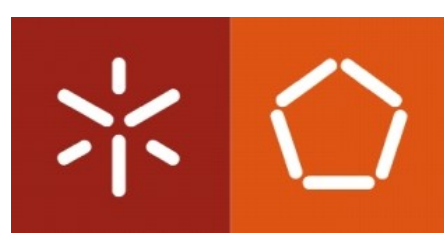
The cryptographic algorithm processes the input elements (bit, byte, or character) of the plaintext continuously, producing output one element at a time.

- They typically use the key as input to a pseudorandom bit generator that produces a stream of elements (e.g., bits or bytes) that are apparently random.
- The output of the generator, called `keystream`, is combined one element at a time with the plaintext stream, e.g., using XOR operation.
- They are similar to the *One-Time Pad*. However, stream ciphers use a pseudorandom number stream.

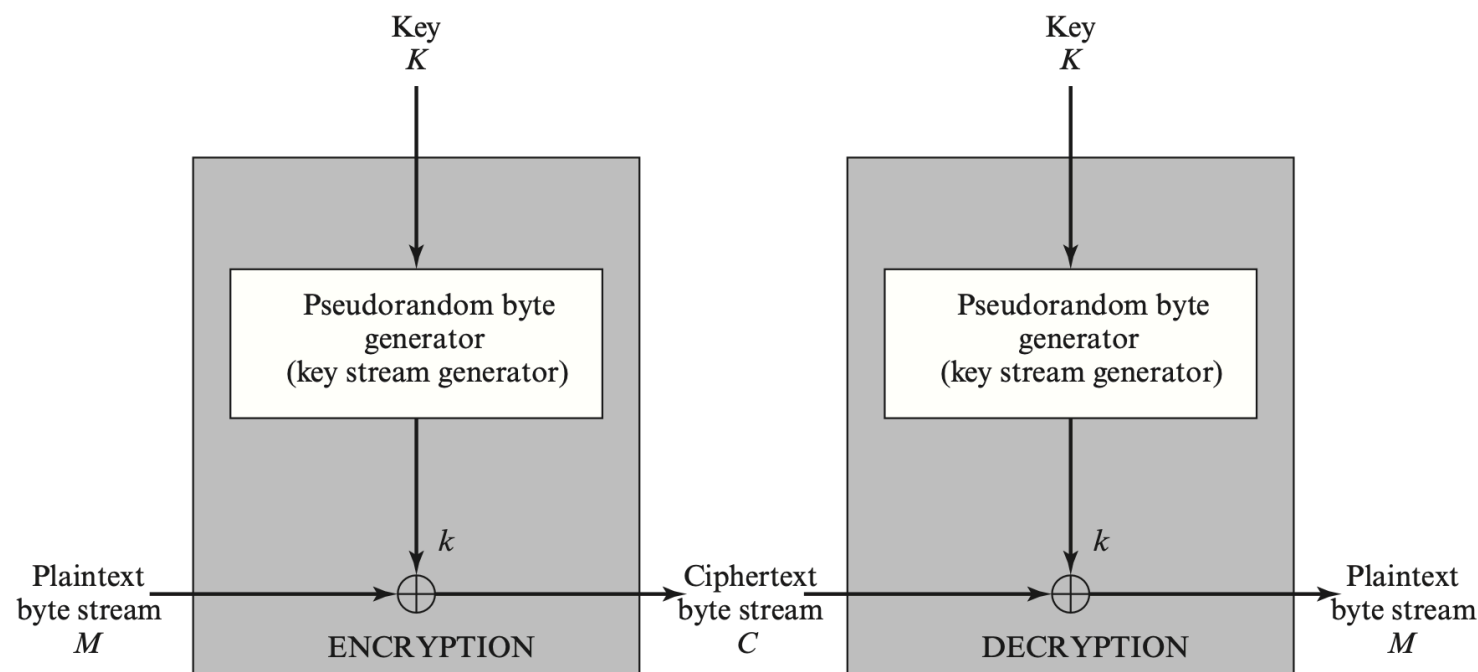


Source: Stallings, W. & Brown, L. Cryptography and Network Security: Principles and Practice

Stream Ciphers

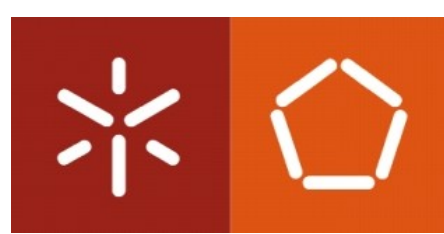


- Synchronization
 - *Synchronous*: the key sequence is independent of the message
 - *Self-synchronizing*: with the ability to recover synchronization even when bits of the cryptogram are lost.



Stream Ciphers

Random Bit Generation



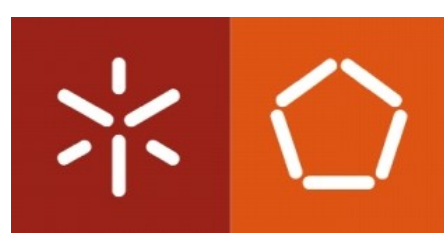
» Important NIST documents

- SP 800-90A: *Recommendation for Random Number Generation Using Deterministic Random Bit Generators*, 2015.
- SP 800-90B: *Recommendation for the Entropy Sources Used for Random Bit Generation*, 2018.
- SP 800-90C: *Recommendation for Random Bit Generator (RBG) Constructions*, 2016.
- SP 800-22: *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications*, 2010.



Stream Ciphers

Random Bit Generation



» The use of Random Numbers

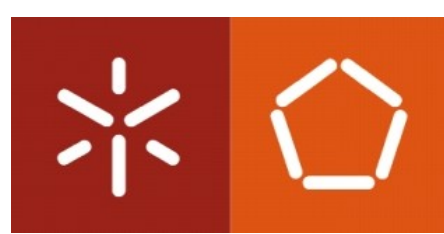
Random binary numbers are used in manifold network security algorithms and protocols based on cryptography.

- Key distribution
- Bidirectional authentication
- Session key generation
- Generation of keys for the RSA public-key encryption algorithm
- Generation of a bit stream for symmetric stream encryption



Stream Ciphers

Random Bit Generation



» Principles and Requirements

Randomness

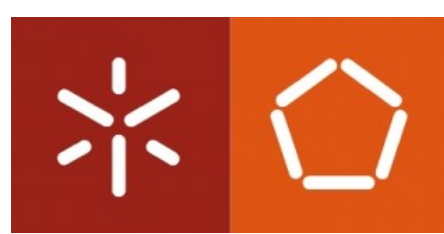
The sequence of numbers has to be random in some well-defined statistical sense. Two criteria are used to validate that:

- **Uniform distribution:** The distribution of bits (zeros and ones) in the sequence should be uniform.
- **Independence:** No one subsequence in the sequence can be inferred from the others.

Unpredictability

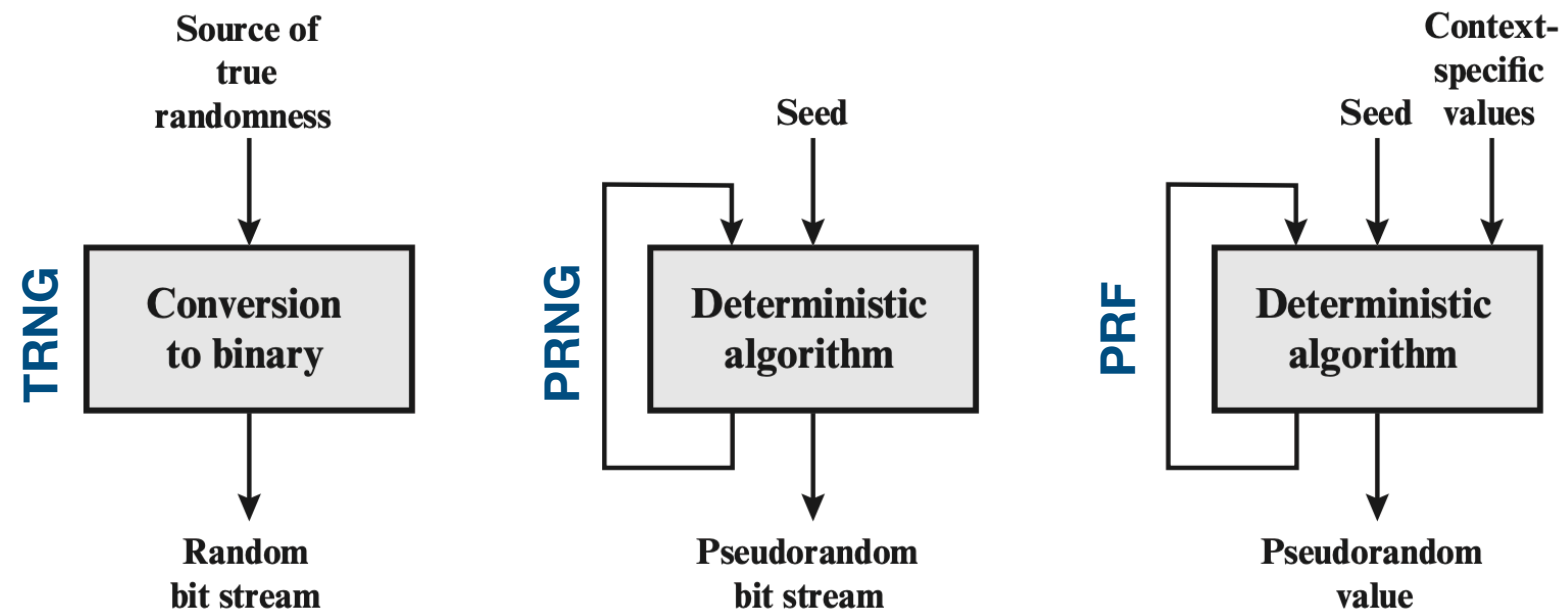
Each number should be statically independent of other numbers in the sequence. Therefore, unpredictable.

Although true random numbers are used in some applications, they have some limitations, such as inefficiency. Thus, it is more common to implement algorithms that generate sequences that appear to be random, *i.e.*, **Pseudorandom numbers**.



Stream Ciphers

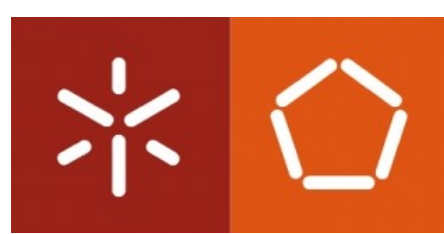
Random & Pseudorandom Generators



- **True Random Number Generator (TRNG):** takes as input a source that is effectively random, *i.e.*, entropy source (e.g., keystroke timing patterns or instantaneous values of the system clock). It may simply involve the conversion of an analog source to a binary output.
- **Pseudorandom Number Generator (PRNG):** a deterministic algorithm that produces an open-ended sequence of bits as input to a symmetric stream cipher.
- **Pseudorandom Function (PRF):** a deterministic algorithm that produces a pseudorandom string of bits of some fixed length. Symmetric encryption keys and nonces are examples of their uses.

Stream Ciphers

Pseudorandom Generators



» Requirements

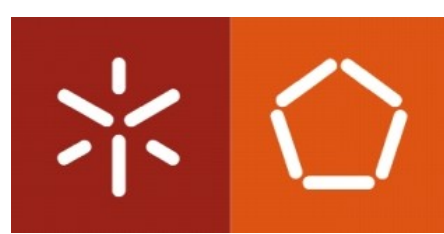
Randomness

No single test can determine if a PRNG/PRF generates numbers that have the characteristic of randomness. The best that can be done is to apply a sequence of tests. If the PRNG/PRF exhibits randomness based on multiple trials, then it can be assumed that it satisfies the requirement. The tests should seek for three characteristics:

- **Uniformity:** At any point in the generation of a sequence of pseudorandom bits, the occurrence probability of zero or one is exactly $1/2$.
- **Scalability:** Any test applicable to a sequence must also be applied to subsequences extracted at random. If a sequence is random, then any subsequence should also be random.
- **Consistency:** The behaviour of a generator must be consistent across starting values (i.e., seeds).

Stream Ciphers

Pseudorandom Generators



» Requirements

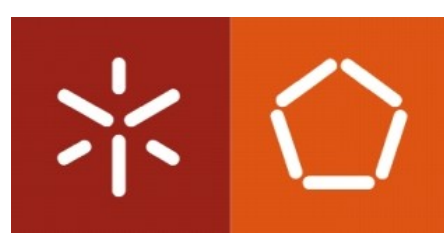
Unpredictability

A stream of pseudorandom numbers should exhibit two forms of unpredictability:

- **Forward unpredictability:** If the `seed` is unknown, the next output bit in the sequence should be unpredictable regardless of any knowledge of previous bits in the sequence.
- **Backward unpredictability:** It should not be feasible to determine the `seed` from knowledge of any generated values. No correlation between a `seed` and any value generated from that seed should be evident.

Stream Ciphers

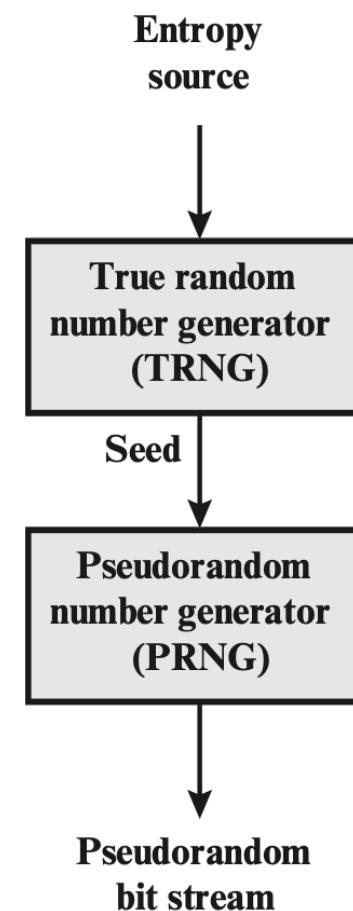
Pseudorandom Generators



» SEED Requirements

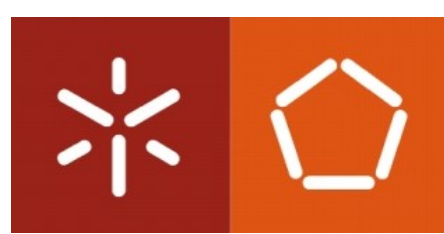
For cryptographic applications, the *seed* that serves as input must be secure.

- PRNG/PRF are deterministic algorithms. The output can also be determined if an adversary can deduce the *seed*.
- The *seed* must also be unpredictable.
- It is usual to use a TRNG to generate *seeds*.



Stream Ciphers

Pseudorandom Generators



» Algorithm design

Purpose-built algorithms

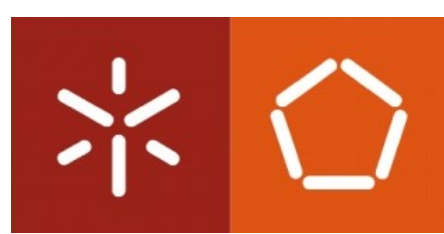
Designed specifically and solely for the purpose of generating pseudorandom bit streams.

- Linear Congruential Generators
- Blum Blum Shub Generator
- For specific stream ciphers, e.g., RC4

Algorithms based on existing cryptographic algorithms

Cryptographic algorithms have the effect of randomizing input data (Confusion & Diffusion)

- Symmetric block ciphers
- Asymmetric ciphers
- Hash functions
- Message Authentication Codes (MAC)



Stream Ciphers

Pseudorandom Generators - Example

» Linear Congruential Generator

The algorithm is parameterized with four numbers,

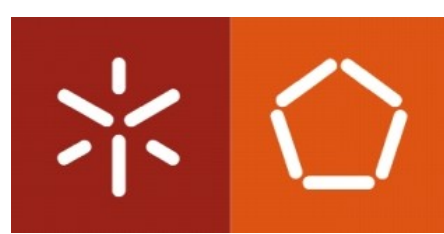
- m the modulus $m > 0$
- a the multiplier $0 < a < m$
- c the increment $0 \leq c < m$
- x_0 the starting value/seed $0 \leq x_0 < m$

The sequence of random numbers $\{x_n\}$ is obtained via the following iterative equation:

$$x_{n+1} = (ax_n + c) \bmod m$$

- The selection of values for a , c , and m is critical for a good random number generator.
- Three tests can be used to evaluate a generator:
 - The function should be a full-period (it should generate all numbers from 0 to $m-1$).
 - The generated sequence should appear random.
 - The function should implement efficiently with 32-bit arithmetic

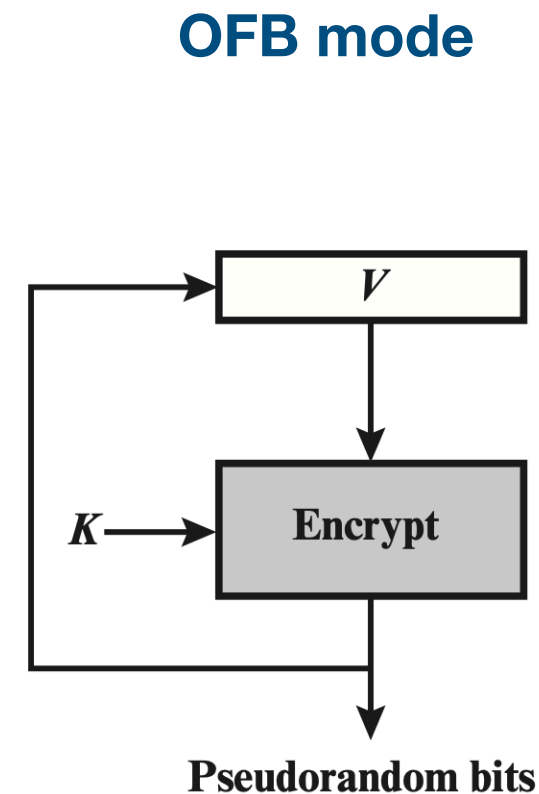
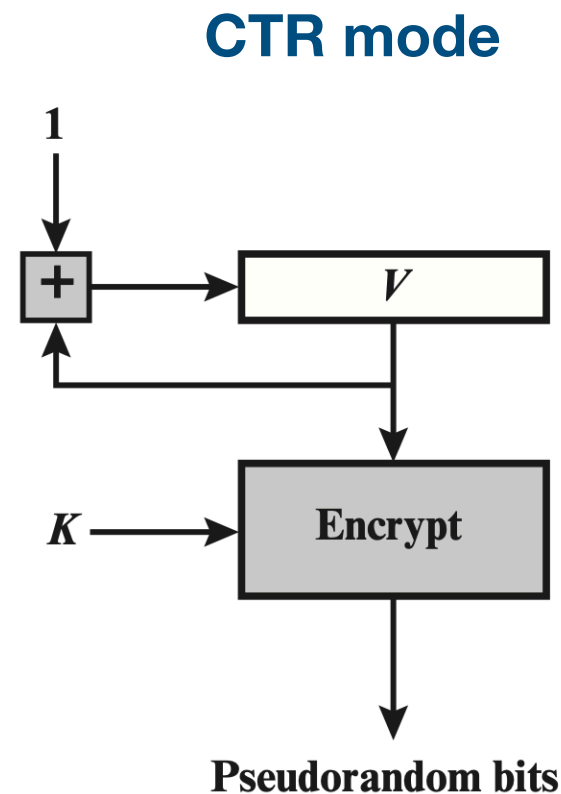
$$x_{n+1} = (ax_n) \bmod (2^{31}-1), \text{ for } c=0$$



Stream Ciphers

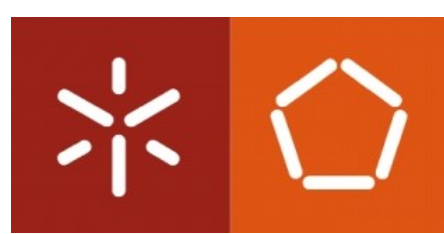
Pseudorandom Generators - Example

➤ Using block cipher modes of operation



What constitutes the seed?

The seed has two parts: the `encryption` key and a value `v`



Stream Ciphers

True Random Number Generators

» Entropy sources

A TRNG uses a nondeterministic source to produce randomness.

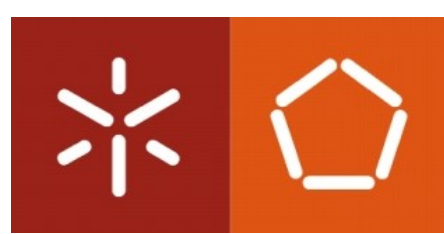
- Thermal noise
- Sound/Video input
- Disk Drives rotations speed fluctuation

Considerations

- TRNG is impractical for stream ciphers, as the sender would need to securely transmit the ciphertext and keystream as plaintext to the receiver.
- Also, the mechanism used to generate true random numbers may not generate bits at the required rate of the application.



Stream Ciphers

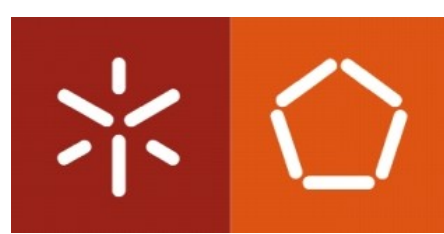


» Comparing PRNG and TRNG

	Pseudorandom Number Generators	True Random Number Generators
Efficiency	Very efficient	Generally inefficient
Determinism	Deterministic	Nondeterministic
Periodicity	Periodic	Aperiodic

Stream Ciphers

RC4

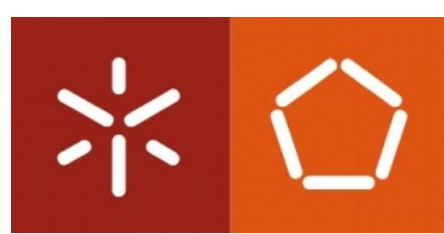


- Designed by Ron Rivest in 1987
- It is a **variable-key-size** stream cipher with **byte-oriented** operations
- The algorithm is based on **random permutations**
- Tests have shown that the period of the cipher is likely to be greater than 10^{100}
- It is efficient in both software and hardware implementations
 - Requires eight to sixteen machine operations
- It is used in:
 - SSL/TLS standards (Secure Sockets Layer / Transport Layer Security)
 - WEP (Wired Equivalent Privacy) protocol
 - WPA (WiFi Protected Access)



Stream Ciphers

RC4



» Algorithm

A variable key of from 1 to 256 bytes is used to initialize a 256-byte state vector **S** with elements $S[0], S[1], \dots, S[255]$

Initialization of **S**

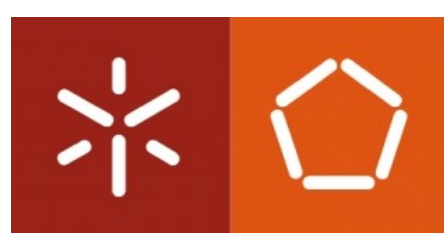
- The entries of **S** are set equal to the values from 0 through 255 in ascending order, *i.e.*, $S[0]=0, S[1]=1, \dots, S[255]$
- The key **K** is copied to a temporary vector **T** of size 256 bytes (if **K** < 256 bytes, **K** is repeated to fill out **T**)
- The **initial permutation** of **S** consists of swapping its values according to a scheme dictated by **T**.

```
/* Initial permutation of S */  
j = 0;  
for i=0 to 255 do  
    j = (j + S[i] + T[i]) mod 256;  
    swap (S[i], S[j]);
```

It is just a permutation of **S** values

Stream Ciphers

RC4



» Algorithm

Stream generation

- Once the S vector is initialized, the input key is no longer used.
- The **stream generation** performs a new permutation of all S elements using a scheme dictated by the current configuration of S .
- After $S[255]$ is reached, the process continues, starting over again at $S[0]$

```
/* Stream generation */
i, j = 0;
while (true)
    i = (i+1) mod 256;
    j = (j+S[i]) mod 256;
    swap (S[i], S[j]);
    t = (S[i]+S[j]) mod 256;
    k = S[t];
```

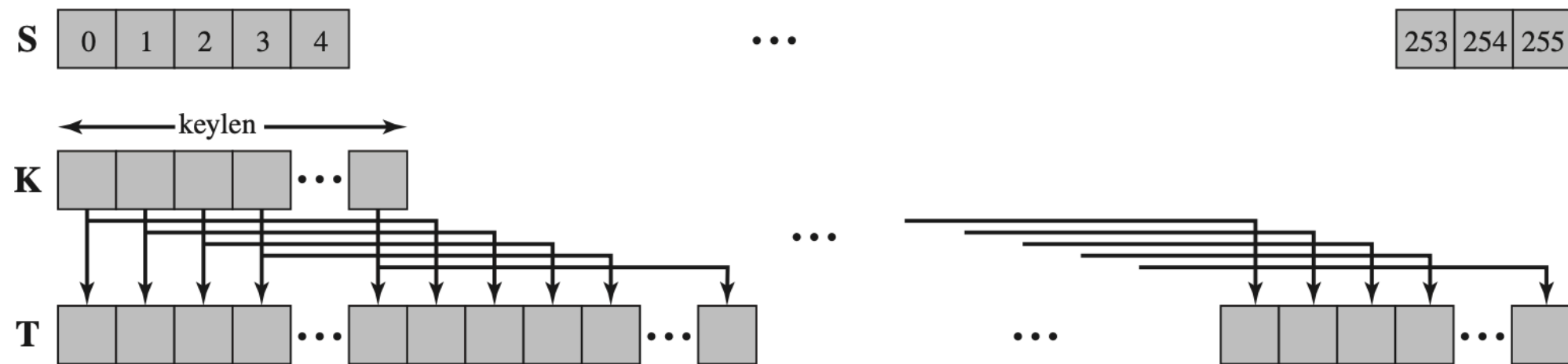
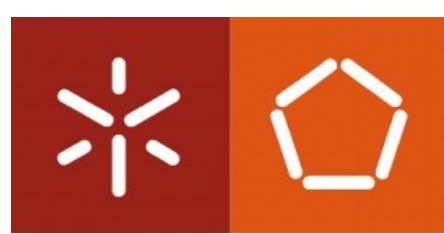
Encryption / Decryption

- To encrypt, XOR the value k with the next byte of plaintext.
- To decrypt, XOR the value k with the next byte of ciphertext.

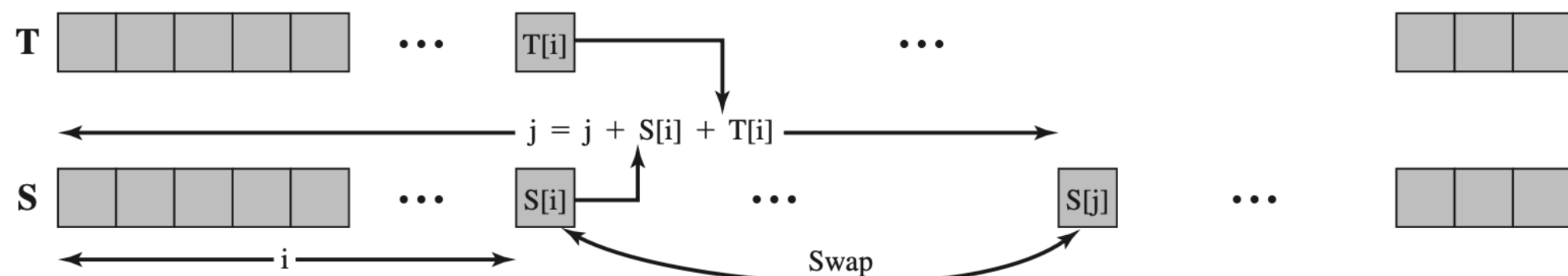


Stream Ciphers

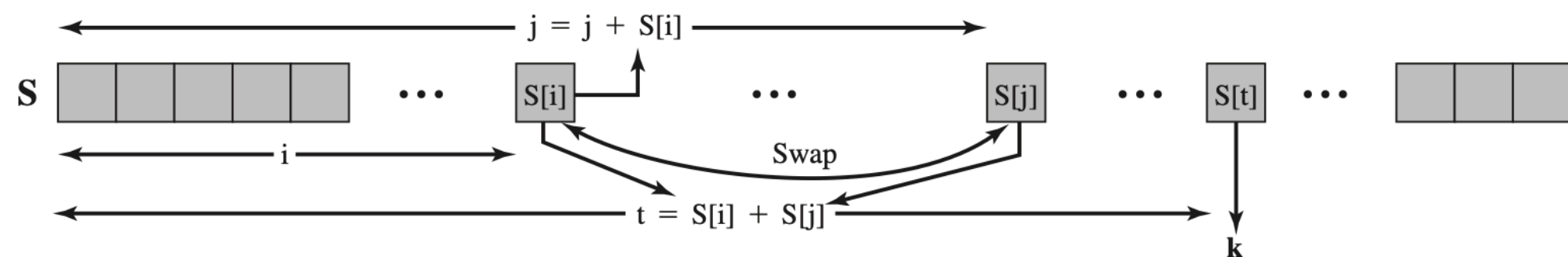
RC4



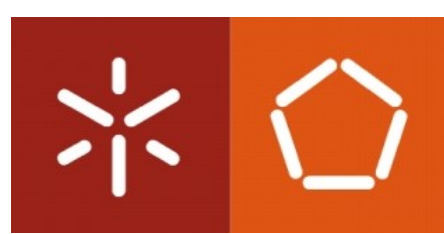
(a) Initial state of S and T



(b) Initial permutation of S



(c) Stream generation



Authenticated Encryption

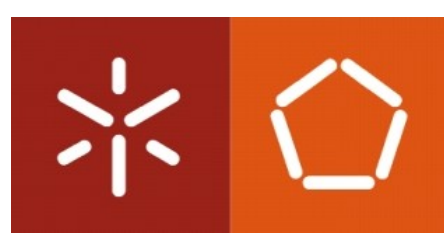
Approaches

Encryption systems that simultaneously protect confidentiality and authenticity (integrity)

» Hashing followed by encryption ($H \rightarrow E$)

1. Compute the cryptographic hash over m as $h = H(m)$
2. Encrypt the message plus the digest as $E(k, (M || h))$





Authenticated Encryption Approaches

➤ Authentication followed by encryption (A → E)

Use two keys

1. Authenticate the plaintext by computing the MAC value: $T = \text{MAC}(K_1, M)$
2. Encrypt the message plus the tag: $E(K_2, [M || T])$

➤ Encryption followed by authentication (E → A)

Use two keys

1. Encrypt the message: $C = E(K_2, M)$
2. Authenticate the ciphertext: $T = \text{MAC}(K_1, C)$

➤ Independent encryption and authentication (E + A)

Use two keys

1. Encrypt the message: $C = E(K_2, M)$
2. Authenticate the plaintext: $T = \text{MAC}(K_1, M)$



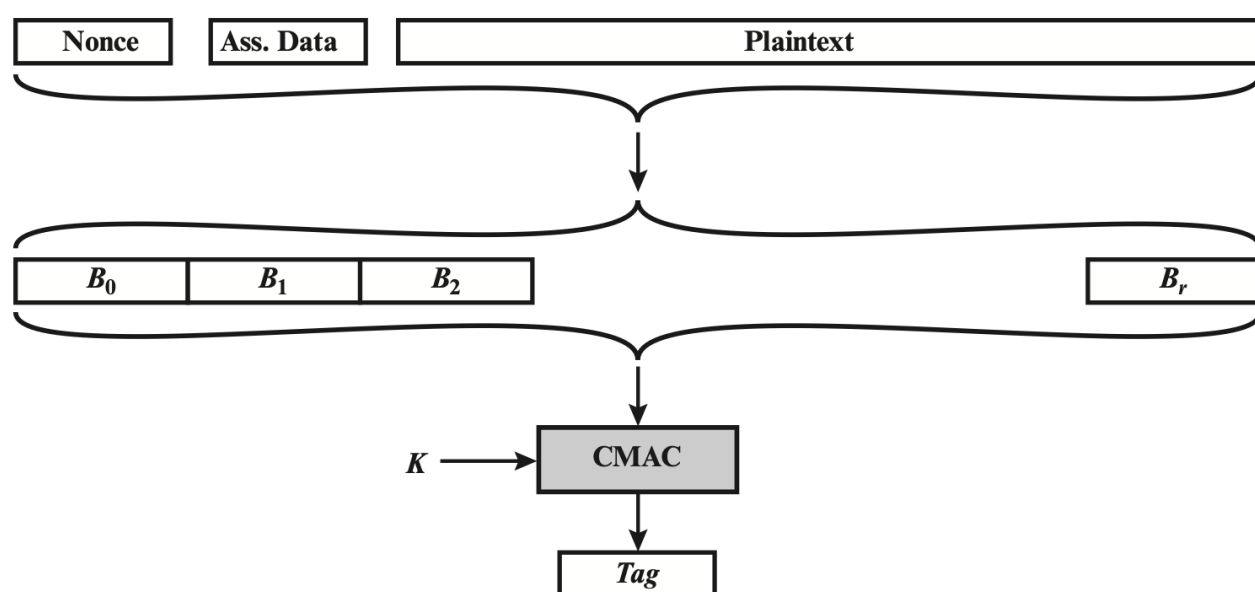


Authenticated Encryption

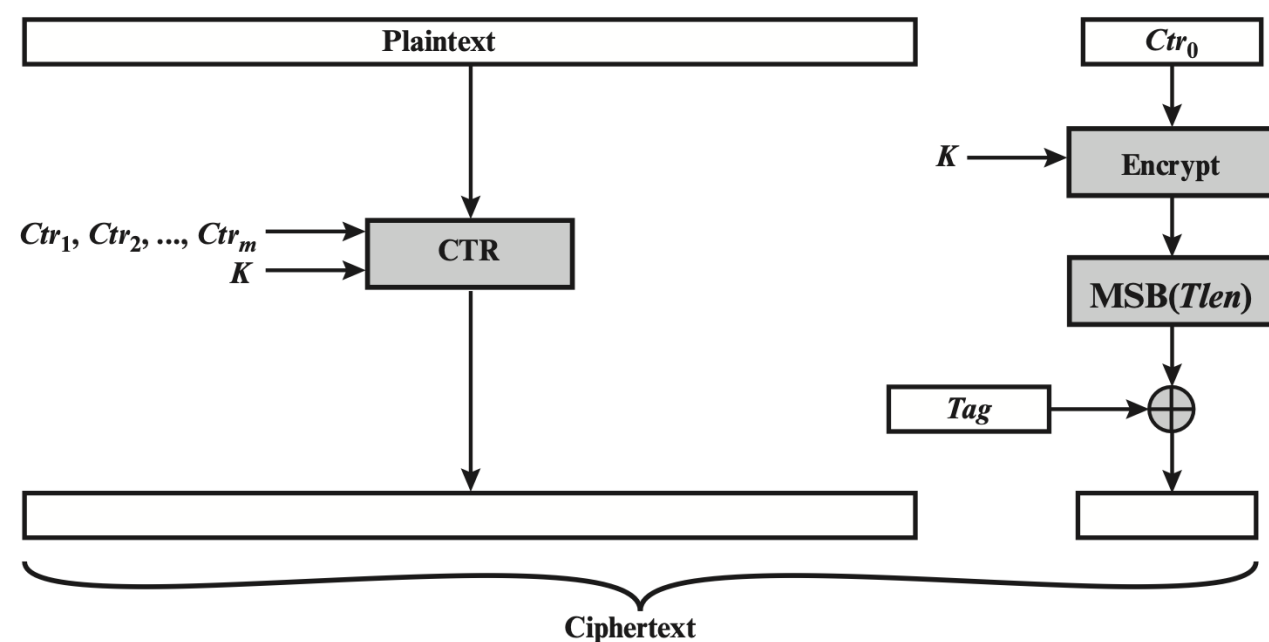
Example: CCM

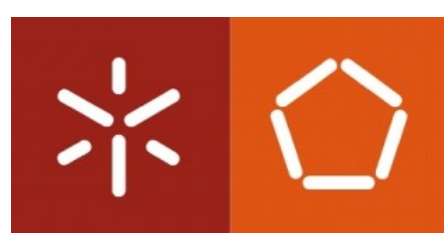
Counter with Cipher Block Chaining-Message Authentication Code

Authentication



Encryption





Practice

Select one block cipher and implement a pseudorandom number generator. What's the difference between your solution and using the operating system's `urandom()` function?





University of Minho
School of Engineering



Cryptography & Network Security

Bachelor in Telecommunications and Informatics Engineering

João Marco Silva

Department of Informatics
joaomarco@di.uminho.pt

2024/2025